

A small hybrid language model under a CPU inference budget

DASE7506 Project 1

29 September 2026

Abstract

I trained a causal language model from scratch on the supplied WikiText-2 training split. The final predictor combines an eight-block attention/convolution network, a copy distribution over the current 256-token window, and an order-six modified Kneser–Ney (MKN) model built only from training text. A small gate mixes the neural and count probabilities. On the complete test split, the frozen model obtains 1.415658 bits per byte (BPB), compared with 2.102015 for the supplied baseline. Local Windows CPU evaluation takes 3.62 times the baseline time and uses 55.81 MB of inference assets. A separate Linux one-thread measurement exceeds the 5-times time limit, so portability to the course machine is a remaining risk.

1 Task and evaluation

The assignment fixes a 2,048-token BPE tokenizer and an evaluator that scores independent causal windows of 256 tokens. At position t , the model predicts x_{t+1} from the observed prefix $x_{\leq t}$, without carrying state between windows. The score is

$$\text{BPB} = \frac{-\sum_t \log_2 p(x_{t+1} \mid x_{\leq t})}{\text{number of raw UTF-8 bytes in the split}}.$$

All model selection used the supplied validation split. I kept the data, tokenizer and evaluator unchanged, used no external training text or pretrained weights, and froze the method before the complete-test evaluation. The course limits are five times baseline CPU scoring time, 4 GiB peak evaluation RAM, and 64 MiB of uncompressed inference assets.

2 Model

The neural backbone has width 288 and eight causal blocks. Blocks 1, 3, 5 and 7 use global self-attention; blocks 2, 4, 6 and 8 use gated causal depthwise convolution with kernel width seven. The model also uses rotary positions, RMSNorm, SwiGLU feed-forward layers, and tied token/output embeddings. This layout keeps some global context mixing while replacing half of the attention blocks with cheaper local mixing.

The output head first forms a 2,048-way vocabulary distribution. A separate pointer attends only to input tokens at or before the current position and adds probability to their token IDs. Its triangular mask rules out later input positions. A sparse order-six MKN model supplies another normalized distribution using within-window token history and count tables built from the training split. The final gate uses four input-derived confidence features: the neural/copy top log-probability, its top-two margin, the MKN backoff at the highest matched order, and that order’s largest continuation mass. If w_t is its bounded sigmoid output, the final distribution is

$$p_t = (1 - w_t)p_{\text{neural+copy},t} + w_t p_{\text{MKN},t}.$$

Neither the gate nor the count model sees the target token during inference. Gate coefficients and feature normalization came from training text; the final feature subset and slope were selected on validation and are not independent test evidence.

The neural lineage used R-Drop during training and later checkpoint averaging, continuation, output calibration, count-aware training and distillation. These auxiliary objectives are absent at inference. The Stage143 release itself did not retrain the model: it stores one OpenVINO FP32 feature graph plus a PyTorch head and sparse count tables, avoiding a duplicate backbone in the checkpoint.

3 Experiments and interpretation

The clearest equal-target comparison uses seed 17, 7,200 updates, batch size 32, the same sampled-window stream, and 58,982,400 primary next-token presentations per run. R-Drop performs two stochastic forward passes, so equal target counts do not mean equal training FLOPs.

Model	Validation BPB	Interpretation
Ordinary objective	1.464994	Reference (Stage26)
Add R-Drop	1.450613	Same network (Stage47)
Hybrid backbone	1.428594	Width also changes (Stage54)

The Stage47–Stage26 contrast isolates the training objective. Stage54 also changes width and feed-forward capacity, so its gain cannot be attributed to convolution alone. Adding a fixed-weight train-only MKN expert to Stage54 reduced complete validation BPB from 1.428594 to 1.423206. Later steps were not equal-budget single-factor controls: a stronger neural checkpoint, order-six counts, calibration and the causal gate together reached 1.399686 validation BPB. With the later Stage92 neural model held fixed, the order-six count model with a fixed mixture weight scored 1.401288; the selected dynamic gate reached 1.399686. This small gate gain was chosen on validation and should not be described as a fresh replication.

This progression separates the effects of training regularization, the backbone and count-model complementarity. It does not establish that this architecture is optimal: the later checkpoint received more training and was selected through repeated validation comparisons. The compact evidence table in `RESULTS.md` links each reported comparison to its original measurement.

Complete test split	BPB	Scored targets
Supplied baseline	2.102015	428,405
Frozen Stage143 predictor	1.415658	428,405

Stage143 reduces test BPB by 32.65% relative to the baseline. Its test BPB is 0.015971 higher than its selected validation BPB. The two splits may differ in difficulty, and repeated development on one validation split can also make that split optimistic; this single comparison cannot separate those explanations. The final test result was not used to change the frozen predictor.

4 Cost, reproducibility and limitations

Three fresh-process Windows CPU FP32 comparisons gave a 3.617702 median-time ratio (86.0511 versus 23.7861 seconds), 2,176,729,088 bytes of maximum peak RSS, and 55,810,412 bytes of conservatively counted inference assets. All three pass on that machine. On a separate two-logical-CPU Linux host, one-thread scoring reproduced the validation BPB but took 5.502555 times its local baseline; a four-thread run could not be measured there. The Windows result therefore does not establish that the course’s CPU will pass the time gate.

The main neural lineage records at least 255,225,110 primary training-target presentations and 7,754.94 seconds of training. Its alternate teacher adds training outside that main path. The archived experiment metrics contain 14.88 hours of comparable recorded training, including

rejected runs. Both figures omit some setup, evaluation and failed-job costs and are lower bounds, not a total hardware bill.

The training recipe in `TRAINING.md` and `code/scripts/reproduce_training.py` covers both teacher branches, prescribed checkpoint averages, train-only count construction and gate fitting. The initial model uses 7,200 updates; its successive neural continuations use 4,800, 4,800, 3,600 and 3,600 updates, followed by five output-bias epochs and 3,600 count-aware updates. An alternate teacher uses 7,200 plus 4,800 updates. Distillation uses 1,800 updates with teacher weights 0.55/0.45 and soft/hard loss weights 0.75/0.25. Retraining produces a new predictor requiring its own validation and resource measurements, not the hash-pinned scored artifact.

The hashes are recorded in the qualification, freeze and test JSON files under `code/results/stage143-evidence/`. From `code/`, install `requirements_stage143.txt` after the base requirements and run:

```
python evaluate.py \
--checkpoint checkpoints/stage143-openvino-order6.pt \
--device cpu --precision fp32 --threads 4 \
--split test --output reproduced-test.json
```

The matching checkpoint and ONNX graph are required; the ZIP manifest records their SHA-256 hashes and the exact release-code commit. No retraining is needed to reproduce the score.

5 Reuse and AI assistance

The course baseline, data, tokenizer and evaluator were supplied. The implementation uses RoPE [1], RMSNorm [2], SwiGLU [3], R-Drop [4], modified Kneser–Ney smoothing [5] and within-context copying [6]; no external pretrained weights or training text were reused. OpenAI Codex substantially assisted with implementation, experiments, debugging, analysis and report writing.

References

- [1] J. Su et al. [RoFormer: Enhanced Transformer with Rotary Position Embedding](#). arXiv:2104.09864, 2021.
- [2] B. Zhang and R. Sennrich. [Root Mean Square Layer Normalization](#). NeurIPS, 2019.
- [3] N. Shazeer. [GLU Variants Improve Transformer](#). arXiv:2002.05202, 2020.
- [4] X. Liang et al. [R-Drop: Regularized Dropout for Neural Networks](#). NeurIPS, 2021.
- [5] S. F. Chen and J. Goodman. [An Empirical Study of Smoothing Techniques for Language Modeling](#). Computer Speech & Language, 13(4):359–394, 1999.
- [6] S. Merity et al. [Pointer Sentinel Mixture Models](#). arXiv:1609.07843, 2016.